“C 语言程序设计实验”实验报告

实验 1—选择与循环结构

1．实验目的

（1）熟悉 vc6.0 IDE 包括源程序编辑、编译、链接以及错误信息（语法错误和语义错误）；
（2）选择结构程序、初步的循环结构程序的练习。

2．实验内容

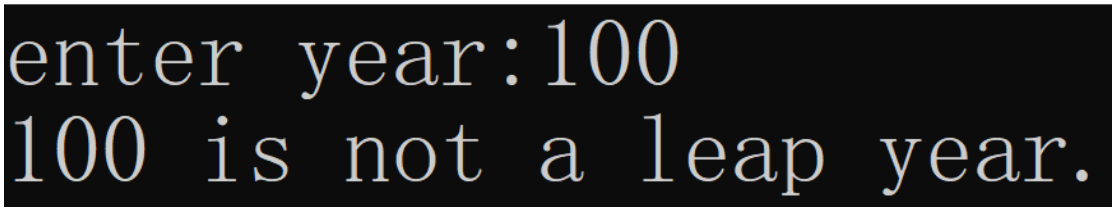
（1）例 4.8 输入年份，判断其是否为闰年。
（2）例 4.9 求 ax2+bx+c=0 的解
（3）统计每个学生 5 门课的平均成绩，学生数为 10。使用键盘输入显示器输出。
（4）第五章习题 8 求水仙花数。 三位数，各位数字立方和等于该数。
（5）(选作) 统计每个学生 5 门课的平均成绩，学生数为 10。使用文本文件输入，显示器和文本文件输出。

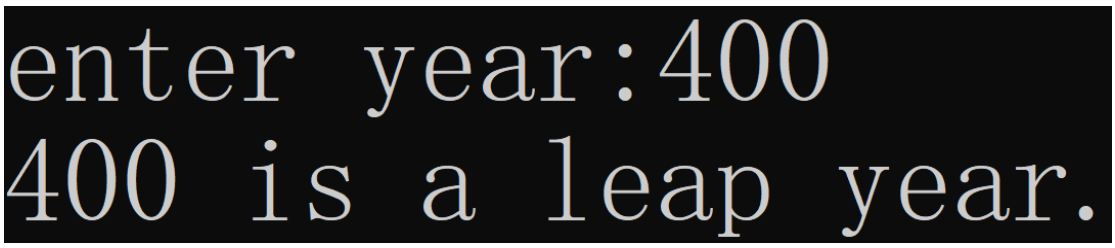
3.程序清单及运行结果

（1）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main()
{
    int year, leap;
    printf("enter year:");
    scanf("%d", &year);
    // 核心判断逻辑开始
    if (year % 4 == 0)
    {
        // 能被4整除时，进一步判断是否能被100整除
        if (year % 100 == 0)
        {
            // 能被100整除时，再判断是否能被400整除
            if (year % 400 == 0)
                // 能被400整除，是闰年
                leap = 1;
            else
                // 能被100整除但不能被400整除，不是闰年
                leap = 0;
        }
        else
        {
            // 能被4整除但不能被100整除，是闰年
            leap = 1;
        }
    }
    else
```

```
    {
        // 不能被4整除，不是闰年
        leap = 0;
    }
    // 核心判断逻辑结束
    if (leap)
        printf("%d is ", year);
    else
        printf("%d is not ", year);
    printf("a leap year.\n");
    return 0;
}
```





（2）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <math.h>
int main()
{
    double a, b, c, disc, x1, x2, realpart, imagpart;
    printf("请输入方程的系数a, b, c（形如ax^2 + bx + c = 0）：\n");
    scanf("%lf%lf%lf", &a, &b, &c);
    // 计算判别式
    disc = b * b - 4 * a * c;
    if (disc > 0)
    {
        // 有两个不同的实根
        x1 = (-b + sqrt(disc)) / (2 * a);
        x2 = (-b - sqrt(disc)) / (2 * a);
        printf("有两个不同的实根：x1 = %lf, x2 = %lf\n", x1, x2);
    }
    else if (disc == 0)
    {
```

```
        // 有两个相等的实根
        x1 = x2 = -b / (2 * a);
        printf("有两个相等的实根：x1 = x2 = %lf\n", x1);
    }
    else
    {
        // 有两个复根
        realpart = -b / (2 * a);
        imagpart = sqrt(-disc) / (2 * a);
        printf("有两个复根：\n");
        printf("x1 = %lf + %lfi\n", realpart, imagpart);
        printf("x2 = %lf - %lfi\n", realpart, imagpart);
    }
    return 0;
}
```

```
请输入方程的系数a, b, c（形如ax^2 + bx + c = 0）：
1
2
1
有两个相等的实根：x1 = x2 = -1.000000
```

```
请输入方程的系数a, b, c（形如ax^2 + bx + c = 0）：
3
1
2
有两个复根：
x1 = -0.166667 + 0.799305i
x2 = -0.166667 - 0.799305i
```

（3）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main()
{
    int i, j;
    // 定义一个二维数组，用于存储10个学生的5门课成绩
    float students[10][5];
    // 定义一个一维数组，用于存储每个学生的平均成绩
    float averages[10];
    // 循环输入10个学生的成绩
    for (i = 0; i < 10; i++)
    {
        // 提示输入第i + 1个学生的成绩
        printf("请输入第 %d 个学生的5门课成绩：\n", i + 1);
```

```
        for (j = 0; j < 5; j++)
        {
            // 输入第i个学生的第j门课成绩
            scanf("%f", &students[i][j]);
        }
    }
    // 计算每个学生的平均成绩
    for (i = 0; i < 10; i++)
    {
        // 初始化每个学生的成绩总和为0
        float sum = 0;
        for (j = 0; j < 5; j++)
        {
            // 累加第i个学生的5门课成绩
            sum += students[i][j];
        }
        // 计算第i个学生的平均成绩
        averages[i] = sum / 5;
    }
    // 输出每个学生的平均成绩
    for (i = 0; i < 10; i++)
    {
        // 输出第i + 1个学生的平均成绩，保留两位小数
        printf("第 %d 个学生的平均成绩是：%.2f\n", i + 1, averages[i]);
    }
    return 0;
}
```

```
请输入第 1 个学生的5门课成绩：
1 1 1 1 1
请输入第 2 个学生的5门课成绩：
2 2 2 2 2
请输入第 3 个学生的5门课成绩：
3 3 3 3 3
请输入第 4 个学生的5门课成绩：
4 4 4 4 4
请输入第 5 个学生的5门课成绩：
5 5 5 5 5
请输入第 6 个学生的5门课成绩：
6 6 6 6 6
请输入第 7 个学生的5门课成绩：
7 7 7 7 7
请输入第 8 个学生的5门课成绩：
8 8 8 8 8
请输入第 9 个学生的5门课成绩：
9 9 9 9 9
请输入第 10 个学生的5门课成绩：
0 0 0 0 0
第 1 个学生的平均成绩是：1.00
第 2 个学生的平均成绩是：2.00
第 3 个学生的平均成绩是：3.00
第 4 个学生的平均成绩是：4.00
第 5 个学生的平均成绩是：5.00
第 6 个学生的平均成绩是：6.00
第 7 个学生的平均成绩是：7.00
第 8 个学生的平均成绩是：8.00
第 9 个学生的平均成绩是：9.00
第 10 个学生的平均成绩是：0.00
```

（4）

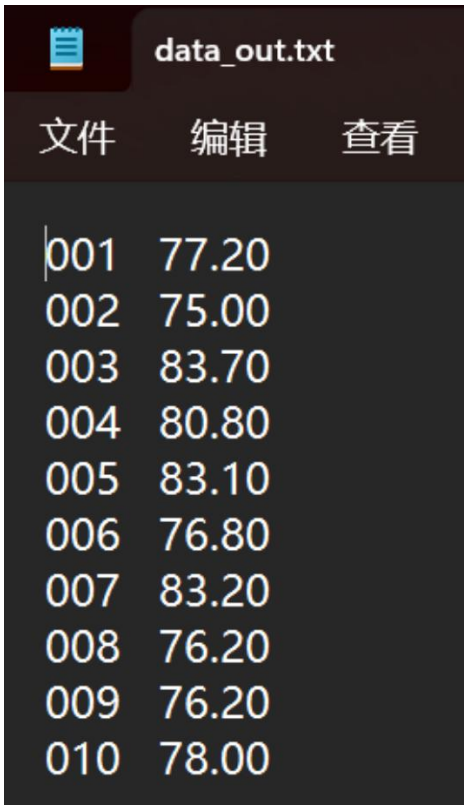
```c
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <math.h>
int main() {
    int num;
    int hundreds, tens, units;
    // 输出提示信息
    printf("三位数的水仙花数有：\n");
    // 循环遍历所有三位数（100到999）
    for (num = 100; num <= 999; num++) {
        // 提取百位数
        hundreds = num / 100;
        // 提取十位数
        tens = (num % 100) / 10;
        // 提取个位数
        units = num % 10;
        // 判断是否为水仙花数
        // pow函数用于计算一个数的立方
        if (pow(hundreds, 3) + pow(tens, 3) + pow(units, 3) == num) {
            // 输出水仙花数
            printf("%d ", num);
        }
    }
    // 换行
    printf("\n");
    return 0;
}
```

```
三位数的水仙花数有：
153 370 371 407
```

（5）

```c
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
main()
{
    float score, ave;
    int i, j, n = 10;
    FILE* fp1, * fp2;
    //以文本文件读方式打开当前工作目录下的data_in.txt文件
    fp1 = fopen("data_in.txt", "r");
    //以文本文件写方式打开工作目录下的data_out.txt文件
    fp2 = fopen("data_out.txt", "w");
```

```
    if (fp1 == NULL || fp2 == NULL)
    {
        printf("open file error!\n"); return 1;
    }
    for (i = 0; i < 10; i++)
    {
        for (j = 0, ave = 0.0; j < 5; j++)
        {
            fscanf(fp1, "%f", &score);
            ave = ave + score;
        }
        ave = ave / 5;
        fprintf(fp2, "%03d  %6.2f\n", i + 1, ave);
    }
    fclose(fp1); fclose(fp2);
    return 0;
}
```



4. 上机遇到的问题及解决办法记录以及体会

“C 语言程序设计实验”实验报告

实验 2—循环与数组

1．试验目的

（1） 练习循环迭代程序设计方法。

（2） 掌握与数组有关的算法（特别是排序算法）。

（3） 掌握字符串函数的使用。

（4） 掌握 VC++6.0 IDE 的程序调试（Debug）手段。

2．实验内容

（1） 第五章习题 14 用牛顿迭代法求方程的根（p141）。$2x^3-4x^2+3x-6=0$ 在 1.5 附近的根。

（2） 用选择法对 10 个整数排序（教材第 6 章第 2 题）。

（3） 找出一个二维数组的“鞍点”（教材第 6 章第 8 题）

（4） 例 6.9 有 3 个字符串要求找出其中最大者。

解题思路（改）：定义三个一维的字符数组 str1、str2、str3,大小为 10。每一数组存放一个字符串 char str1[10],str2[10],str3[10];

```
if (strcmp(str1,str2)>0)
    strcpy(string,str1);
else
    strcpy(string,str2);
if (strcmp(str3,string)>0)
    strcpy(string,str3);
```

（5） （选作）输入文件中含有 10 个同学名字及三门课程成绩的数据，编写一个程序将文件的数据读入，并按总分从高到低排序，将姓名 总分 三门课分数输出到显示器和一个输出文件。

输入文件样例

章三 56 78 89 （名字无空格）

。。。

输出样例

。。。

章三 56 78 89 223

。。。

3.程序清单及运行结果

（1）

```
#include<stdio.h>
#include<math.h>
int main()
{
    // 声明变量，x0和x1用于迭代计算方程的根，f是方程的值，f1是方程的导数的值
    double x0, x1, f, f1;
    // 初始化x1为1.5，这是根的初始猜测值
```

```
    x1 = 1.5;
    // 开始迭代计算
    do
    {
        // 将x1的值赋给x0，因为下一次迭代需要用上一次的x1作为新的x0
        x0 = x1;
        // 计算方程2x^3 - 4x^2 + 3x - 6在x0处的值
        f = 2 * x0 * x0 * x0 - 4 * x0 * x0 + 3 * x0 - 6;
        // 计算方程2x^3 - 4x^2 + 3x - 6的导数(6x^2 - 8x + 3)在x0处的值
        f1 = 6 * x0 * x0 - 8 * x0 + 3;
        // 使用牛顿迭代法公式x1 = x0 - f(x0)/f'(x0)更新x1的值
        x1 = x0 - f / f1;
        // 当x1和x0的差的绝对值大于等于1e-5时，继续迭代
    } while (fabs(x1 - x0) >= 1e-5);
    // 输出方程在1.5附近的根
    printf("此方程在1.5附近的根为%lf", x1);
    return 0;
}
```

```
此方程在1.5附近的根为2.000000
```

（2）

```
#include<stdio.h>
int main()
{
    // 声明变量，i和j用于循环，min用于记录最小值的下标，a[11]用于存储输入的数据
    int i, j, min, a[11];
    // 提示用户输入10个数据
    printf("请输入10个数据：");
    // 循环读取10个数据并存储到数组a中
    for (i = 1; i <= 10; i++)
    {
        // 提示输入每个元素
        printf("a[%d]=", i);
        // 读取数据并存入数组a
        scanf_s("%d", &a[i]);
    }
    printf("\n");
    // 外层循环，控制比较轮数，进行9轮比较（n - 1轮，n为数据个数）
    for (i = 1; i <= 9; i++)
    {
        // 假设当前下标为i的元素是最小值，记录其下标
        min = i;
        // 内层循环，从i + 1开始比较，找到最小值的下标
        for (j = i + 1; j <= 10; j++)
        {
```

```
            // 如果找到比当前最小值更小的元素，更新最小值的下标
            if (a[min] > a[j])
            {
                min = j;
            }
        }
        // 使用异或操作交换a[i]和a[min]的值
        a[i] = a[i] ^ a[min];
        a[min] = a[i] ^ a[min];
        a[i] = a[i] ^ a[min];
    }
    // 输出排序后的数列
    printf("\n排序后的数列为：");
    for (i = 1; i <= 10; i++)
    {
        printf("%5d", a[i]);
    }
    return 0;
}
```

```
请输入10个数据：a[1]=1
a[2]=2
a[3]=3
a[4]=4
a[5]=5
a[6]=6
a[7]=7
a[8]=8
a[9]=9
a[10]=0

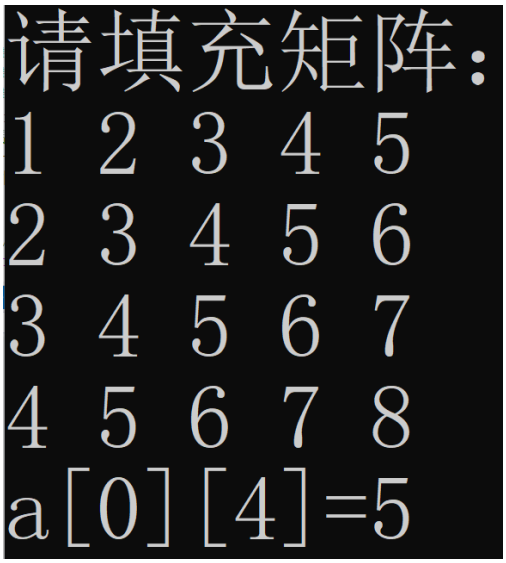
排序后的数列为：    0    1    2    3    4    5    6    7    8    9
```

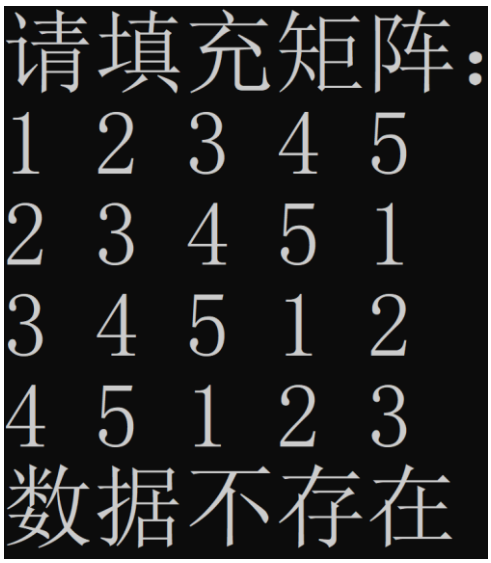
（3）

```
#include<stdio.h>
// 定义矩阵的行数为4
#define N 4
// 定义矩阵的列数为5
#define M 5
int main()
{
    // 声明循环变量i、j、k，用于矩阵遍历
    // 声明max用于存储每行的最大值，maxj用于存储最大值所在的列索引
    // 声明flag用于标记是否找到鞍点
    int i, j, k, a[N][M], max, maxj, flag;
    // 提示用户填充矩阵
    printf("请填充矩阵：\n");
```

```
// 外层循环遍历矩阵的行
for (i = 0; i < N; i++)
{
    // 内层循环遍历矩阵的列
    for (j = 0; j < M; j++)
    {
        // 读取用户输入的矩阵元素
        scanf_s("%d", &a[i][j]);
    }
}
// 外层循环遍历矩阵的每一行
for (i = 0; i < N; i++)
{
    // 初始化当前行的最大值为该行的第一个元素
    max = a[i][0];
    // 初始化当前行最大值所在的列索引为0
    maxj = 0;
    // 内层循环遍历当前行的所有元素
    for (j = 0; j < M; j++)
    {
        // 如果找到比当前最大值更大的元素
        if (a[i][j] > max)
        {
            // 更新当前行的最大值
            max = a[i][j];
            // 更新最大值所在的列索引
            maxj = j;
        }
    }
    // 标记当前行的最大值可能是鞍点
    flag = 1;
    // 遍历矩阵的每一行，检查当前行的最大值是否为所在列的最小值
    for (k = 0; k < N; k++)
    {
        // 如果当前行的最大值大于所在列的某个元素
        if (max > a[k][maxj]) {
            // 标记当前值不是鞍点
            flag = 0;
            // 继续检查下一行
            continue;
        }
    }
    // 如果当前值是鞍点
    if (flag)
```

```
            {
                // 输出鞍点的位置和值
                printf("a[%d][%d]=%d\n", i, maxj, max);
                // 找到一个鞍点后，跳出循环
                break;
            }
        }
        // 如果没有找到鞍点
        if (!flag)
        {
            // 输出提示信息
            printf("数据不存在\n");
        }
        return 0;
}
```

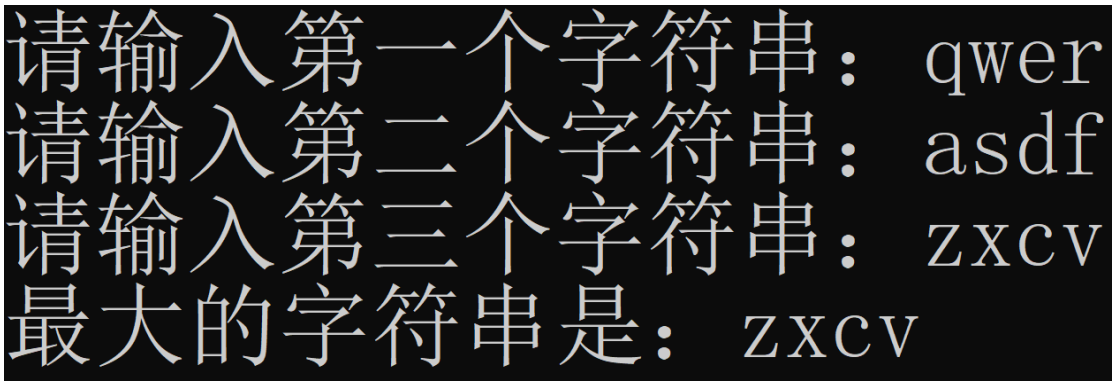




（4）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[10], str2[10], str3[10];
    char string[10];
    // 输入三个字符串
    printf("请输入第一个字符串：");
    scanf("%s", str1);
    printf("请输入第二个字符串：");
    scanf("%s", str2);
    printf("请输入第三个字符串：");
    scanf("%s", str3);
```

```
    // 比较str1和str2，将较大的字符串复制到string
    if (strcmp(str1, str2) > 0)
    {
        strcpy(string, str1);
    }
    else
    {
        strcpy(string, str2);
    }
    // 比较str3和string，将较大的字符串复制到string
    if (strcmp(str3, string) > 0)
     {
        strcpy(string, str3);
    }
    // 输出最大的字符串
    printf("最大的字符串是：%s\n", string);
    return 0;
}
```

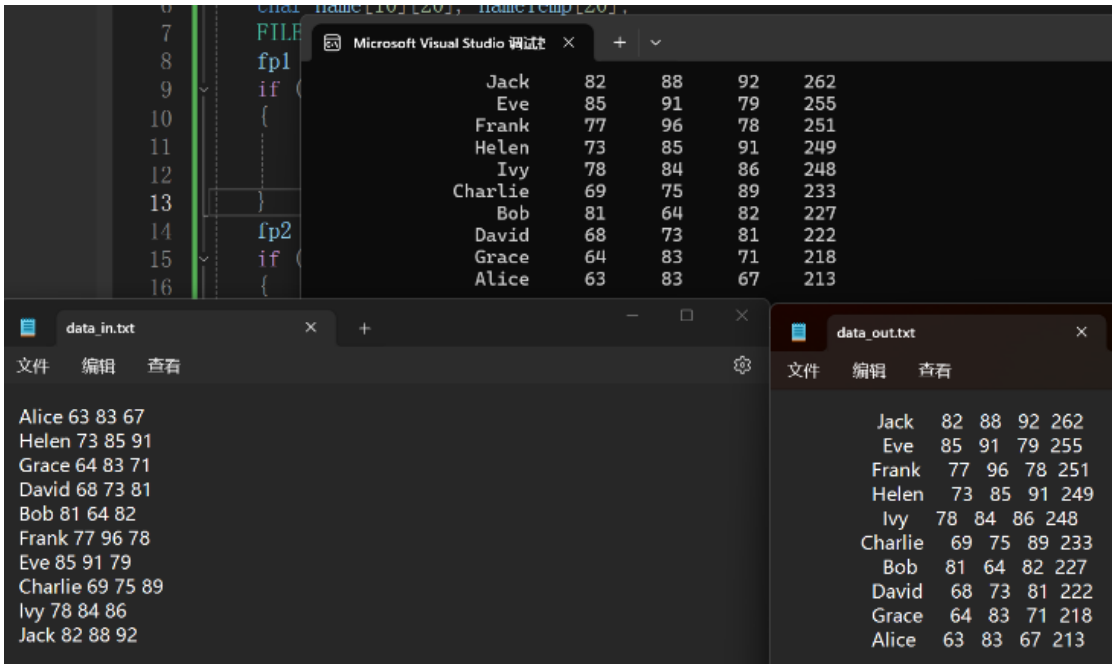


（5）

```
#define _CRT_SECURE_NO_WARNINGS
#include <string.h>
#include <stdio.h>
int main() {
    int s[10][4], sTemp[4], i, j, k;
    char name[10][20], nameTemp[20];
    FILE* fp1, * fp2;
    fp1 = fopen("data_in.txt", "r");
    if (fp1 == 0)
    {
        printf("Cannot open file data_in.txt\n");
        return 1;
    }
    fp2 = fopen("data_out.txt", "w");
    if (fp2 == 0)
```

```
{
    printf("Cannot open file data_out.txt\n");
    return 1;
}
for (i = 0; i < 10; i++)
{
    s[i][3] = 0;
    fscanf(fp1, "%s", &(name[i][0]));
    for (j = 0; j < 3; j++)
    {
        fscanf(fp1, "%d", &(s[i][j]));
        s[i][3] = s[i][3] + s[i][j];
    }
}
for (i = 0; i < 10 - 1; i++)
{
    for (j = 10 - 1; j > i; j--)
    {
        if (s[j][3] > s[j - 1][3])
        {
            for (k = 0; k < 4; k++)
            {
                sTemp[k] = s[j - 1][k];
                s[j - 1][k] = s[j][k];
                s[j][k] = sTemp[k];
            }
            strcpy(nameTemp, &(name[j - 1][0]));
            strcpy(&(name[j - 1][0]), &(name[j][0]));
            strcpy(&(name[j][0]), nameTemp);
        }
    }
}
for (i = 0; i < 10; i++)
{
    fprintf(fp2, "%20s  ", &(name[i][0]));
    printf("%20s  ", &(name[i][0]));
    for (k = 0; k < 4; k++)
    {
        fprintf(fp2, "%5d", s[i][k]);
        printf("%5d  ", s[i][k]);
    }
    fprintf(fp2, "\n");
    printf("\n");
}
```

```
    fclose(fp1);
    fclose(fp2);
    return 0;
}
```



4. 上机遇到的问题及解决办法记录以及体会

“C 语言程序设计实验”实验报告

实验 3—函数与变量的存储类型

1. 试验目的

（1） 熟悉函数定义、声明与调用。

（2） 掌握字符串函数的编写。

（3） 掌握自动变量、全局变量、内部静态变量的使用。

（4） 继续掌握 VC++6.0 IDE 的程序调试（Debug）手段

2. 实验内容

（1） 写一个判别素数的函数，在主函数输入输入一个整数，输出是否素数的信息（教材第 7 第 3 题）。
（2） 写一个函数，将一个字符串的元音字母复制到另一个字符串并输出（教材第 7 第 7 题）。函数原型 void cpy(char s1[],char s2[])
（3） 例 7.16 考察局部静态变量的值。
（4） 例 7.14 全局变量的使用，有一个一维数组，内放 10 个学生的成绩，写一个函数当主调函数调用此函数后，能求出平均分、最高分和最低分。定义全局变量数组。
（5） （选作）输入文件中含有 10 个同学名字及三门课程成绩的数据，编写一个程序将文件的数据读入，并按总分从高到低排序，将姓名 总分 三门课分数输出到显示器和一个输出文件。
要求 从文件读入数据、排序、数据输出的功能分别用函数完成。
输入文件样例
章三 56 78 89 （名字无空格）
。。。
输出样例
。。。
章三 56 78 89 223
。。。

3. 程序清单及运行结果

（1）

```
#include <stdio.h>
// 函数声明，pri函数用于判断一个数是否为素数
int pri(int j);
int main()
{
    int j;
    // 提示用户输入一个整数
    printf("请输入一个整数：");
    // 读取用户输入的整数
    scanf_s("%d", &j);
    // 调用pri函数判断j是否为素数
    if (pri(j))
```

```
    {
        // 如果pri函数返回1，说明j是素数，输出相应信息
        printf("%d是素数\n", j);
    }
    else
    {
        // 如果pri函数返回0，说明j不是素数，输出相应信息
        printf("%d不是素数\n", j);
    }
    return 0;
}
// pri函数定义，用于判断一个整数是否为素数
int pri(int j)
{
    // 如果j小于等于1，直接返回0，因为1及以下的数不是素数
    if (j <= 1)
    {
        return 0;
    }
    // 如果j小于等于3，返回1，2和3是素数
    if (j <= 3)
    {
        return 1;
    }
    // 如果j能被2或3整除，说明j不是素数，返回0
    if (j % 2 == 0 || j % 3 == 0)
    {
        return 0;
    }
    // 初始化i为5，用于后续的循环判断
    int i = 5;
    // 当i的平方小于等于j时，继续循环
    while (i * i <= j) {
        // 如果j能被i或i + 2整除，说明j不是素数，返回0
        if (j % i == 0 || j % (i + 2) == 0)
        {
            return 0;
        }
        // i每次增加6，因为大于3的素数都可以表示为6k ± 1的形式
        i += 6;
    }
    // 如果循环结束没有返回0，说明j是素数，返回1
    return 1;
}
```

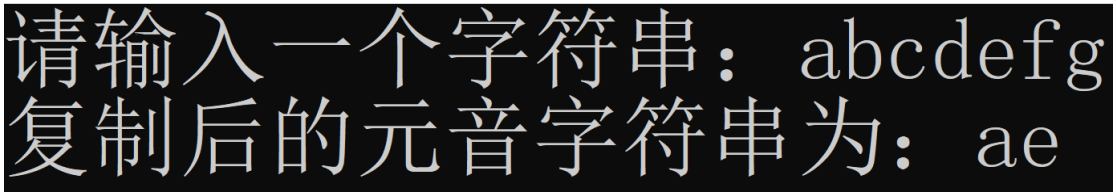




（2）

```
#include <stdio.h>
// 函数声明，cpy函数用于从一个字符串中提取元音字符并复制到另一个字符串
void cpy(char s1[], char s2[]);
// cpy函数定义，实现从s1中提取元音字符并复制到s2
void cpy(char s1[], char s2[])
{
    int i, j; // 定义循环变量i和j
    j = 0; // 初始化目标字符串s2的索引
    i = 0; // 初始化源字符串s1的索引
    // 当s1[i]不是换行符'\n'时，继续循环
    while (s1[i] != '\n')
    {
        // 判断s1[i]是否为元音字母（包括大写和小写）
        if ((s1[i] == 'a' || s1[i] == 'e' || s1[i] == 'i' || s1[i] == 'o' || s1[i] == 'u')
||
            (s1[i] == 'A' || s1[i] == 'E' || s1[i] == 'I' || s1[i] == 'O' || s1[i] == 'U'))
        {
            // 如果是元音字母，将其复制到s2中
            s2[j] = s1[i];
            j++; // 移动目标字符串s2的索引
        }
        i++; // 移动源字符串s1的索引
    }
    s2[j] = '\0'; // 在目标字符串s2的末尾添加字符串结束符
}
int main()
{
    // 定义两个字符数组，用于存储输入的字符串和复制的元音字符串
    char str1[100], str2[100];
```

```
    // 提示用户输入一个字符串
    printf("请输入一个字符串：");
    // 从标准输入读取一行字符串，最多读取99个字符，存储到str1中
    fgets(str1, 100, stdin);
    // 调用cpy函数，从str1中提取元音字符并复制到str2
    cpy(str1, str2);
    // 输出复制后的元音字符串
    printf("复制后的元音字符串为：%s\n", str2);
    return 0;
}
```



（3）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
// 计算每个学生的平均分
void studentAverage(int scores[][5], int numStudents)
{
    // 遍历每个学生
    for (int i = 0; i < numStudents; i++)
    {
        int sum = 0;
        // 计算该学生的总分
        for (int j = 0; j < 5; j++)
        {
            sum += scores[i][j];
        }
        // 输出该学生的平均分
        printf("学生 %d: %.2f\n", i + 1, (float)sum / 5);
    }
}
// 计算每门课的平均分
void courseAverage(int scores[][5], int numStudents)
{
    // 遍历每门课程
    for (int j = 0; j < 5; j++)
    {
        int sum = 0;
        // 计算该门课程的总分
        for (int i = 0; i < numStudents; i++)
        {
```

```
            sum += scores[i][j];
        }
        // 输出该门课程的平均分
        printf("课程 %d: %.2f\n", j + 1, (float)sum / numStudents);
    }
}
// 找出最高分数及其对应的学生和课程
void findHighestScore(int scores[][5], int numStudents)
{
    int highestScore = scores[0][0];
    int studentIndex = 0;
    int courseIndex = 0;
    // 双重循环遍历所有分数
    for (int i = 0; i < numStudents; i++)
    {
        for (int j = 0; j < 5; j++)
        {
            // 如果找到更高的分数
            if (scores[i][j] > highestScore)
            {
                highestScore = scores[i][j];
                studentIndex = i;
                courseIndex = j;
            }
        }
    }
    // 输出最高分数及其对应的学生和课程
    printf("最高分数是 %d,对应的学生是 %d,对应的课程是 %d\n", highestScore, studentIndex
+ 1, courseIndex + 1);
}
int main()
{
    int scores[10][5];
    // 输入学生成绩
    printf("请输入10个学生5门课的成绩：\n");
    for (int i = 0; i < 10; i++)
    {
        for (int j = 0; j < 5; j++)
        {
            scanf("%d", &scores[i][j]);
        }
    }
    studentAverage(scores, 10);
    courseAverage(scores, 10);
```

```
    findHighestScore(scores, 10);
    return 0;
}
```

```
请输入10个学生5门课的成绩：
1 1 1 1 1 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4 5 5 5 5 5 6 6 6 6 6 7 7 7 7 7 8 8 8 8 8 9 9 9 9 9 0 0 0 0 0
学生 1: 1.00
学生 2: 2.00
学生 3: 3.00
学生 4: 4.00
学生 5: 5.00
学生 6: 6.00
学生 7: 7.00
学生 8: 8.00
学生 9: 9.00
学生 10: 0.00
课程 1: 4.50
课程 2: 4.50
课程 3: 4.50
课程 4: 4.50
课程 5: 4.50
最高分数是 9，对应的学生是 9，对应的课程是 1
```

（4）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <ctype.h>
// 将十六进制字符串转换为十进制整数
int hexToDec(char hex[])
{
    int len = strlen(hex);
    int decimal = 0;
    // 遍历十六进制字符串
    for (int i = 0; i < len; i++)
    {
        char digit = hex[i];
        int value;
        // 根据字符判断其十六进制值
        if (isdigit(digit))
        {
            value = digit - '0';
        }
        else if (digit >= 'a' && digit <= 'f')
        {
            value = digit - 'a' + 10;
        }
        else if (digit >= 'A' && digit <= 'F')
        {
            value = digit - 'A' + 10;
        }
        // 根据位置计算十进制值并累加
        decimal += value * (int)pow(16, len - 1 - i);
```

```
    }
    return decimal;
}
int main()
{
    char hex[100];
    // 输入十六进制数
    printf("请输入一个十六进制数：");
    scanf("%s", hex);
    // 转换并输出十进制数
    int decimal = hexToDec(hex);
    printf("对应的十进制数是：%d\n", decimal);
    return 0;
}
```

```
请输入一个十六进制数：99
对应的十进制数是：153
```

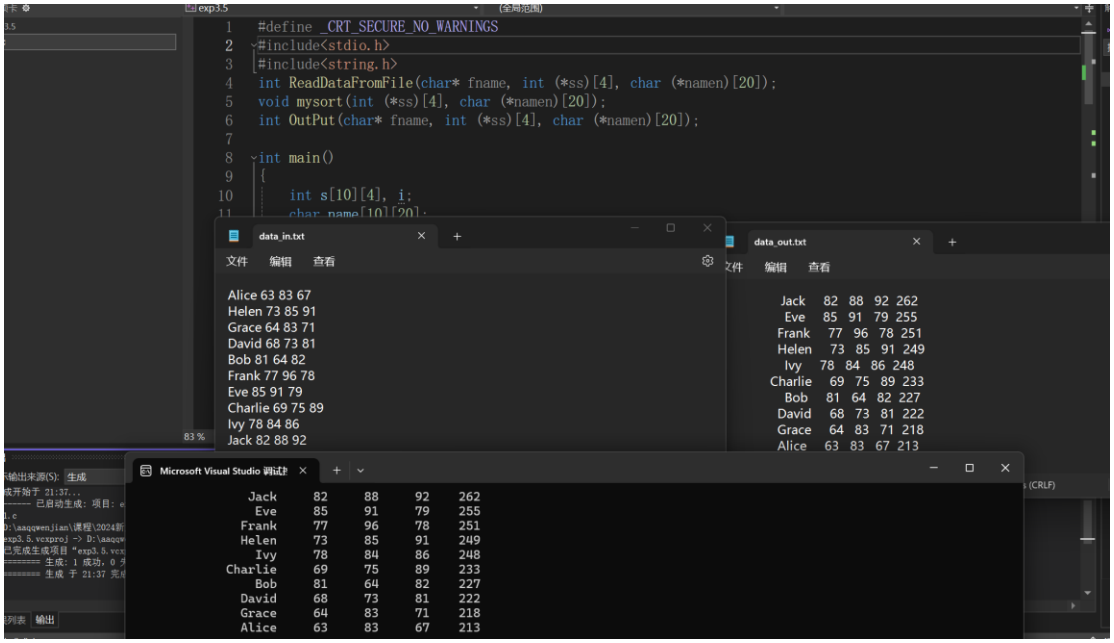
（5）

```
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
#include<string.h>
int ReadDataFromFile(char* fname, int (*ss)[4], char (*namen)[20]);
void mysort(int (*ss)[4], char (*namen)[20]);
int OutPut(char* fname, int (*ss)[4], char (*namen)[20]);

int main()
{
    int s[10][4], i;
    char name[10][20];
    if (ReadDataFromFile("data_in.txt", s, name) == 0) return 0;
    mysort(s, name);
    OutPut("data_out.txt", s, name);
    return 0;
}
int ReadDataFromFile(char* fname, int (*ss)[4], char (*namen)[20])
{
    int i, j;
    FILE* fp1;
    fp1 = fopen(fname, "r");
    if (fp1 == NULL)
    {
        printf("Cannot open file input.txt\n");
```

```
        return 0;
    }
    for (i = 0; i < 10; i++)
    {
        ss[i][3] = 0;
        fscanf(fp1, "%s", &(namen[i][0]));
        for (j = 0; j < 3; j++)
        {
            fscanf(fp1, "%d", &(ss[i][j]));
            ss[i][3] = ss[i][3] + ss[i][j];
        }
    }
    fclose(fp1);
    return 1;
}
void mysort(int (*ss)[4], char (*namen)[20])
{
    int i, j, k;     int sTemp[4];
    char nameTemp[20];
    for (i = 0; i < 10 - 1; i++)
    {
        for (j = 10 - 1; j > i; j--)
        {
            if (ss[j][3] > ss[j - 1][3])
            {
                for (k = 0; k < 4; k++)
                {
                    sTemp[k] = ss[j - 1][k];
                    ss[j - 1][k] = ss[j][k];
                    ss[j][k] = sTemp[k];
                }
                strcpy(nameTemp, &(namen[j - 1][0]));
                strcpy(&(namen[j - 1][0]), &(namen[j][0]));
                strcpy(&(namen[j][0]), nameTemp);
            }
        }
    }
    return;
}
int OutPut(char* fname, int (*ss)[4], char (*namen)[20])
{
    FILE* fp2;  int i, k, r = 1;
    fp2 = fopen(fname, "w");
    if (fp2 == 0)
```

```
    {
        printf("Cannot open file data_out.txt\n");
        r = 0;
    }
    for (i = 0; i < 10; i++)
    {
        if (fp2) fprintf(fp2, "%20s  ", &(namen[i][0]));
        printf("%20s  ", &(namen[i][0]));
        for (k = 0; k < 4; k++)
        {
            if (fp2) fprintf(fp2, "%5d", ss[i][k]);
            printf("%5d  ", ss[i][k]);
        }
        if (fp2) fprintf(fp2, "\n");
        printf("\n");
    }
    if (fp2 != 0) fclose(fp2);
    return r;
}
```



4. 上机遇到的问题及解决办法记录以及体会

“C 语言程序设计实验”实验报告

实验 4—指针

1. 试验目的

（1） 掌握指针的用法。

2. 实验内容

（1） 指针作为函数参数 li0806、li0807。写一个程序，对 10 个整数用冒泡法升序排序，调用上面的 swap 函数对两变量进行内容交换。

（2） 数组名做实参，指针做形参。**例 li0812** 返回某个数组中最大元素的函数。**例 li0813** 将数字字符串转换为整数的函数 my_atoi

（3） 字符串处理，**li0826** 字符串连接。

（4） 内存的动态分配 **li0827**

（5） （选作）读写二进制文件。输入文本文件中含有 10 个同学名字及三门课程成绩的数据，编写一个程序将文件的数据读入，以二进制文件的形式输出，再将该二进制文件读入，并按总分从高到低排序，将姓名 总分 三门课分数输出到显示器和一个二进制输出文件。

要求 从文件读入数据、排序、数据输出的功能分别用函数完成。

文本输入文件样例

章三 56 78 89 （名字无空格）

。。。

提示：

```
int a[10][4],i ,j; char name[10][20];
unsigned int sizei,sizec;
FILE *fpb1,*fpb2;
…
sizei=sizeof(int); sizec=sizeof(char);
fpb1=fopen("outputb.dat","wb");
for(i=0;i<10;i++)
  {
  for(j=0;j<20;j++) fwrite(&name[i][j],sizec,1,fpb1);
  for(j=0;j<4;j++) fwrite(&a[i][j],sizei,1,fpb1);
```

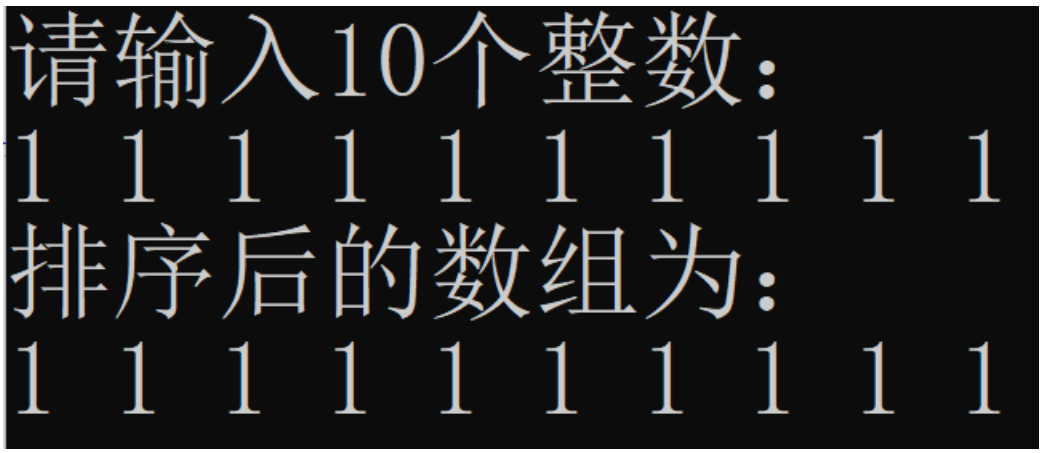
```
  }
fclose(fpb1);
…
fpb1=fopen("outputb.dat","rb");
for(i=0;i<10;i++)
  {
  for(j=0;j<20;j++) fread(&name[i][j],sizec,1,fpb1);
  for(j=0;j<4;j++) fread(&a[i][j],sizei,1,fpb1);
  }
fclose(fpb1);
```

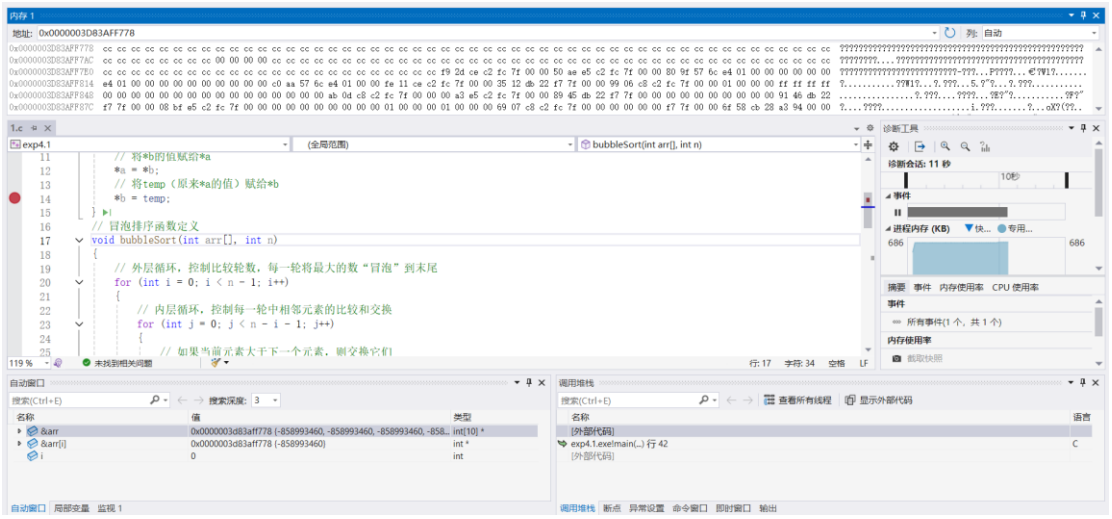
3.程序清单及运行结果

（1）

```c
#include <stdio.h>
// 函数声明：交换两个整数的函数
void swap(int* a, int* b);
// 函数声明：冒泡排序函数
void bubbleSort(int arr[], int n);
// 交换两个整数的函数定义
void swap(int* a, int* b)
{
    // 定义临时变量temp，存储*a的值
    int temp = *a;
    // 将*b的值赋给*a
    *a = *b;
    // 将temp（原来*a的值）赋给*b
    *b = temp;
}
// 冒泡排序函数定义
void bubbleSort(int arr[], int n)
{
    // 外层循环，控制比较轮数，每一轮将最大的数“冒泡”到末尾
    for (int i = 0; i < n - 1; i++)
    {
```

```
        // 内层循环，控制每一轮中相邻元素的比较和交换
        for (int j = 0; j < n - i - 1; j++)
        {
            // 如果当前元素大于下一个元素，则交换它们
            if (arr[j] > arr[j + 1])
            {
                // 调用swap函数交换相邻元素
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}
int main()
{
    int arr[10];
    // 提示用户输入10个整数
    printf("请输入10个整数：\n");
    // 循环读取用户输入的10个整数并存入数组
    for (int i = 0; i < 10; i++)
    {
        scanf_s("%d", &arr[i]);
    }
    // 调用冒泡排序函数对数组进行排序
    bubbleSort(arr, 10);
    // 输出排序后的数组
    printf("排序后的数组为：\n");
    for (int i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```



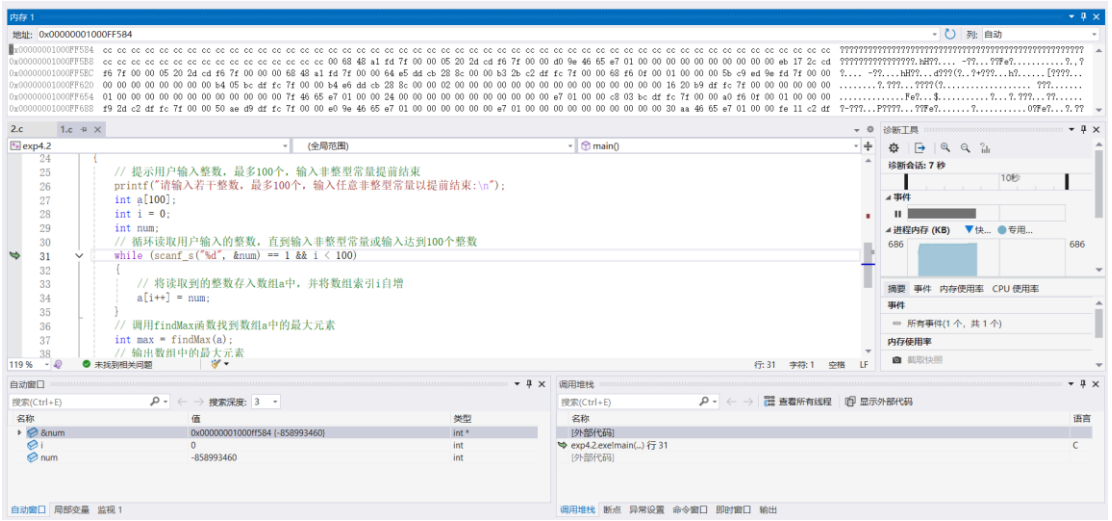


（2）

```c
#include <stdio.h>
// 函数声明：用于在整数数组中找到最大元素
int findMax(int* arr);
// 函数定义：在给定的整数数组中找到最大元素
int findMax(int* arr)
{
    // 初始化max为数组的第一个元素
    int max = *arr;
    // 遍历数组从第二个元素开始到第100个元素
    for (int i = 1; i < 100; i++)
    {
        // 如果当前元素大于max
        if (*(arr + i) > max)
        {
            // 更新max为当前更大的元素
            max = *(arr + i);
        }
    }
    // 返回找到的最大元素
    return max;
}
int main()
{
    // 提示用户输入整数，最多100个，输入非整型常量提前结束
    printf("请输入若干整数，最多100个，输入任意非整型常量以提前结束:\n");
    int a[100];
    int i = 0;
    int num;
    // 循环读取用户输入的整数，直到输入非整型常量或输入达到100个整数
    while (scanf_s("%d", &num) == 1 && i < 100)
```

```
    {
        // 将读取到的整数存入数组a中，并将数组索引i自增
        a[i++] = num;
    }
    // 调用findMax函数找到数组a中的最大元素
    int max = findMax(a);
    // 输出数组中的最大元素
    printf("数组中的最大元素是：%d\n", max);
    return 0;
}
```

```
请输入若干整数，最多100个，输入任意非整型常量以提前结束:
1 2 3 4 5 6 7 8 9 0 1.1
数组中的最大元素是：9
```



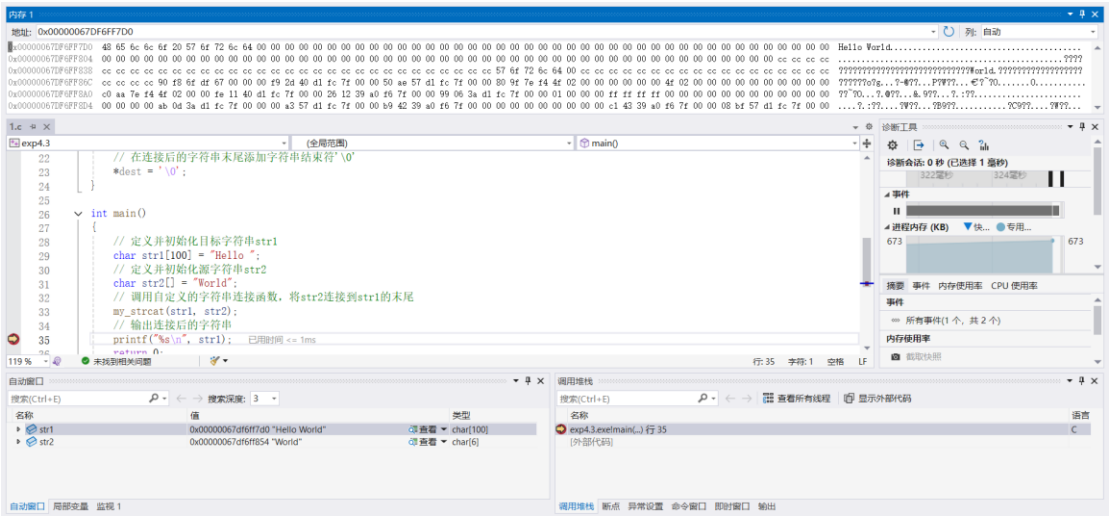
（3）

```
#include <stdio.h>
// 函数声明：自定义的字符串连接函数
void my_strcat(char* dest, char* src);
// 自定义的字符串连接函数定义
void my_strcat(char* dest, char* src)
{
    // 移动指针dest到目标字符串的末尾（即'\0'的位置）
    while (*dest != '\0')
    {
        dest++;
    }
    // 将源字符串src的内容逐个复制到目标字符串dest的末尾
    while (*src != '\0')
    {
        // 将src当前指向的字符复制到dest当前指向的位置
        *dest = *src;
        // 移动dest指针到下一个位置
```

```
        dest++;
        // 移动src指针到下一个位置
        src++;
    }
    // 在连接后的字符串末尾添加字符串结束符'\0'
    *dest = '\0';
}

int main()
{
    // 定义并初始化目标字符串str1
    char str1[100] = "Hello ";
    // 定义并初始化源字符串str2
    char str2[] = "World";
    // 调用自定义的字符串连接函数，将str2连接到str1的末尾
    my_strcat(str1, str2);
    // 输出连接后的字符串
    printf("%s\n", str1);
    return 0;
}
```





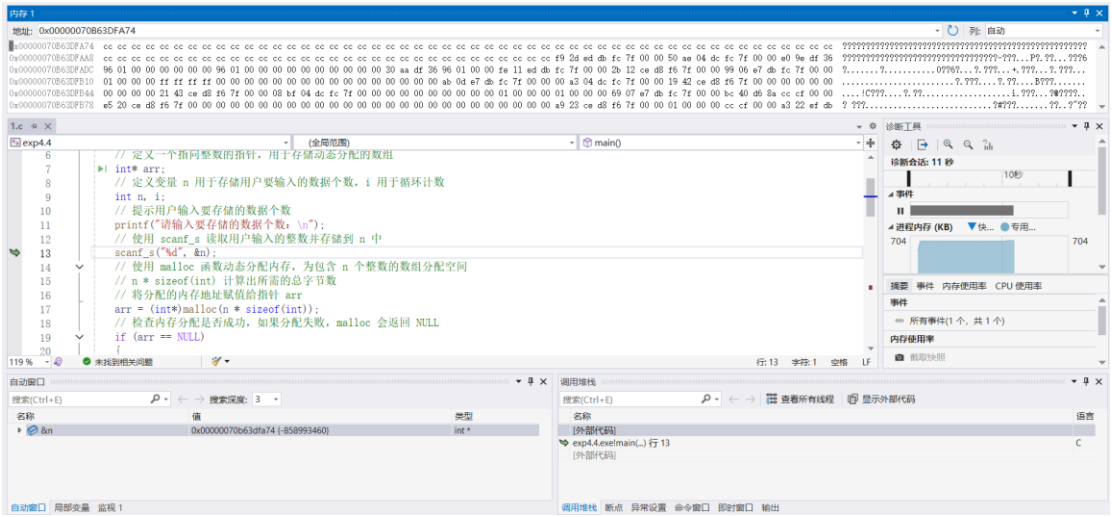
（4）

```
#include <stdio.h>
#include <stdlib.h>
// 主函数，程序的入口
int main()
{
```

```c
// 定义一个指向整数的指针，用于存储动态分配的数组
int* arr;
// 定义变量 n 用于存储用户要输入的数据个数，i 用于循环计数
int n, i;
// 提示用户输入要存储的数据个数
printf("请输入要存储的数据个数：\n");
// 使用 scanf_s 读取用户输入的整数并存储到 n 中
scanf_s("%d", &n);
// 使用 malloc 函数动态分配内存，为包含 n 个整数的数组分配空间
// n * sizeof(int) 计算出所需的总字节数
// 将分配的内存地址赋值给指针 arr
arr = (int*)malloc(n * sizeof(int));
// 检查内存分配是否成功，如果分配失败，malloc 会返回 NULL
if (arr == NULL)
{
    // 如果内存分配失败，打印错误信息
    printf("内存分配失败！\n");
    // 以 -1 作为返回值，表示程序异常结束
    return -1;
}
// 提示用户输入 n 个数据
printf("请输入 %d 个数据：\n", n);
// 使用 for 循环读取用户输入的 n 个整数，并存储到数组 arr 中
for (i = 0; i < n; i++)
{
    // 使用 scanf_s 读取用户输入的整数并存储到 arr[i] 中
    scanf_s("%d", &arr[i]);
}
// 打印提示信息，表明接下来要输出用户输入的数据
printf("输入的数据为：\n");
// 使用 for 循环遍历数组 arr，并输出每个元素的值
for (i = 0; i < n; i++)
{
    // 输出数组元素，每个元素后跟着一个空格
    printf("%d ", arr[i]);
}
// 输出一个换行符，使输出格式更美观
printf("\n");
// 使用 free 函数释放动态分配的内存，避免内存泄漏
free(arr);
// 将指针 arr 赋值为 NULL，防止悬空指针
arr = NULL;
// 程序正常结束，返回 0
return 0;
```

}

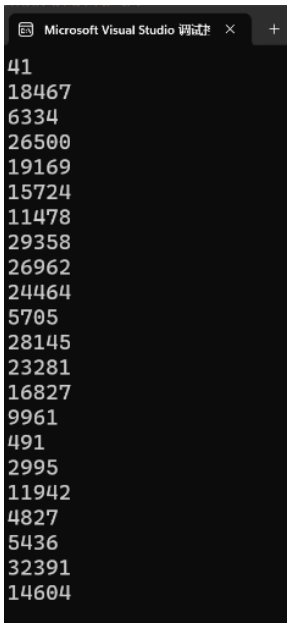
```
请输入要存储的数据个数：
3
请输入 3 个数据：
1
2
3
输入的数据为：
1 2 3
```

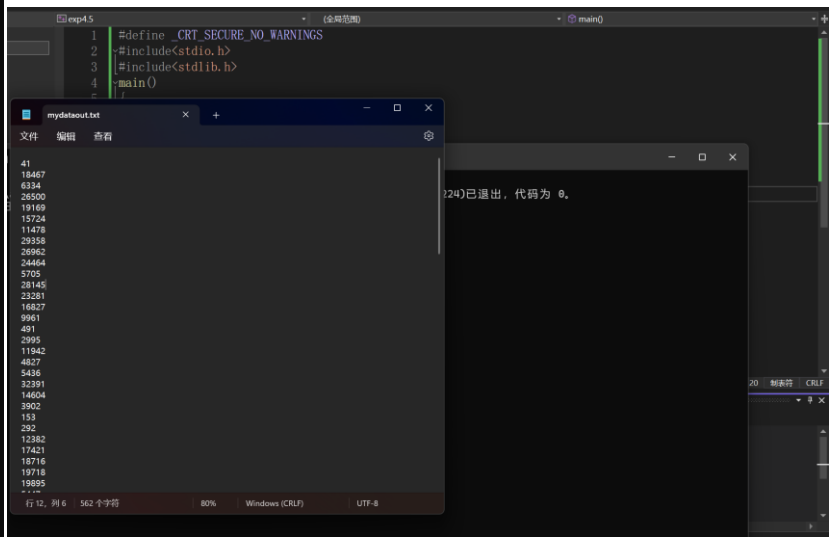


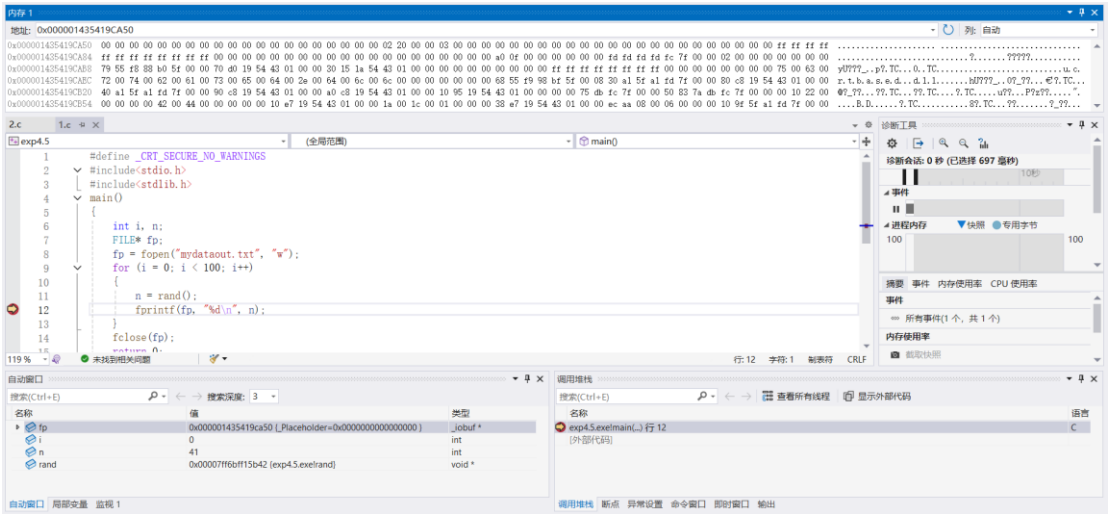
（5）

```
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
#include<stdlib.h>
main()
{
    int i, n;
    FILE* fp;
    fp = fopen("mydataout.txt", "w");
    for (i = 0; i < 100; i++)
    {
```

```
        n = rand();
        fprintf(fp, "%d\n", n);
    }
    fclose(fp);
    return 0;
}
```







4. 上机遇到的问题及解决办法记录以及体会

“C 语言程序设计实验”实验报告

实验 5—结构体与链表

1．试验目的

(1) 掌握结构体的用法。

2. 实验内容

（1） 例 9.2 输入两个学生的学号、姓名和成绩，输出成绩较高学生的学号、姓名和成绩。

（2） 结构体数组。例 9.4 有 n 个学生的信息(包括学号、姓名、成绩)，要求按照成绩的高低顺序输出各学生的信息。

（3） 结构体指针。例 9.5 通过指向结构体变量的指针变量输出结构体变量中成员的信息。

（4） 用指针处理链表。例 9.9 写一函数建立一个有 3 名学生数据的单向动态链表。例 9.10 编写一个输出链表的函数 print。

（5）（选作）分别将（2）用文本文件、二进制文件存储，并读入。

3.程序清单及运行结果

（1）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main() {
    // 定义一个结构体类型 Student，用于存储学生的信息
    // 包含学号（整数）、姓名（字符数组）和成绩（双精度浮点数）
    struct Student
    {
        int num;
        char name[20];
        double score;
    } student1, student2; // 声明两个 Student 类型的变量 student1 和 student2
    // 提示用户输入两名同学的学号、姓名和成绩
    printf("请输入两名同学学号姓名成绩：\n");
    // 使用 scanf 函数读取第一个学生的学号、姓名和成绩
    // &student1.num 表示取 student1 的 num 成员的地址，用于存储读取的学号
    // student1.name 本身就是数组名，代表数组首地址，用于存储读取的姓名
    // &student1.score 表示取 student1 的 score 成员的地址，用于存储读取的成绩
    scanf("%d%s%lf", &student1.num, student1.name, &student1.score);
```

```
    // 同理，读取第二个学生的学号、姓名和成绩
    scanf("%d%s%lf", &student2.num, student2.name, &student2.score);
    // 提示用户即将输出成绩较高者的信息
    printf("成绩较高者为：\n");
    // 判断第一个学生的成绩是否大于第二个学生的成绩
    if (student1.score > student2.score)
        // 如果是，输出第一个学生的学号、姓名和成绩
        printf("%d %s %lf", student1.num, student1.name, student1.score);
    // 判断第一个学生的成绩是否小于第二个学生的成绩
    else if (student1.score < student2.score)
        // 如果是，输出第二个学生的学号、姓名和成绩
        printf("%d %s %lf", student2.num, student2.name, student2.score);
    else
    {
        // 如果两个学生成绩相等，输出第一个学生的信息
        printf("%d %s %lf", student1.num, student1.name, student1.score);
        // 输出第二个学生的信息
        printf("%d %s %lf", student2.num, student2.name, student2.score);
    }
    return 0;
}
```

```
请输入两名同学学号姓名成绩：
1hhx 100,2hhj 99
成绩较高者为：
1 hhx 100.000000
```

（2）

```
#include <stdio.h>
// 定义结构体 Student，用于存储学生信息
// 包含学号（整数）、姓名（单个字符，这里可能定义有误，一般姓名是字符串，应该用字符数组）、成绩（双精度浮点数）
struct Student
{
    int num;
    char name; // 这里应该用 char name[20]; 之类的字符数组来存储姓名更合适
    double score;
};
// 主函数，程序的入口
int main()
{
    // 定义并初始化一个包含 5 个 Student 结构体变量的数组 stu
    // 这里每个学生的姓名部分只存储了一个字符，与实际需求不符
    struct Student stu[5] = {{10101, 'z', 78},{10103, 'w', 98.5},{10106, 'l', 86},{10108, 'l', 73.5},{10110,'s', 100}};
```

```
    // 定义一个临时的 Student 结构体变量，用于交换数据
    struct Student temp;
    // 定义常量 n，表示学生的数量
    const int n = 5;
    // 定义循环变量 i、j、k
    int i, j, k;
    // 提示用户即将输出分数较高者的信息
    printf("分数较高者依次为：\n");
    // 使用选择排序算法对学生数组按照成绩从高到低排序
    for (i = 0; i < n - 1; i++) {
        // 假设当前索引 i 对应的元素是最大值的索引
        k = i;
        // 遍历 i 之后的元素
        for (j = i + 1; j < n; j++)
            // 如果找到比当前最大值还大的元素
            if (stu[j].score > stu[k].score)
                // 更新最大值的索引
                k = j;
        // 交换当前元素（索引 i）和找到的最大值元素（索引 k）
        temp = stu[k];
        stu[k] = stu[i]; // 这里原代码是 stu[k] = stu[j]; 是错误的，应该交换 stu[k] 和
stu[i]
        stu[i] = temp;
    }
    // 遍历排序后的学生数组，输出每个学生的信息
    for (i = 0; i < n; i++)
        // 格式化输出学生的学号、姓名、成绩
        // 这里由于 name 定义为单个字符，输出格式与预期可能不符
        printf("%6d %8c %6.2f\n", stu[i].num, stu[i].name, stu[i].score);
    printf("\n");
    return 0;
}
```

```
分数较高者依次为:
 10110        s 100.00
 10103        W  98.50
 10106        l  86.00
 10101        z  78.00
 10108        l  73.50
```

（3）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>
```

```c
int main() {
    // 定义一个结构体类型 Student，用于存储学生信息
    // 包含学号（长整型）、姓名（字符数组，长度为 20）、性别（字符）和成绩（单精度浮点数）
    struct Student {
        long num;
        char name[20];
        char sex;
        float score;
    };
    // 声明一个 Student 类型的变量 stu_1
    struct Student stu_1;
    // 声明一个指向 Student 类型的指针 p
    struct Student* p;
    // 让指针 p 指向变量 stu_1 的地址
    p = &stu_1;
    // 给 stu_1 的成员 num 赋值为 10101
    stu_1.num = 10101;
    // 使用 strcpy 函数将字符串 "Li Lin" 复制到 stu_1 的 name 数组中
    strcpy(stu_1.name, "Li Lin");
    // 给 stu_1 的成员 sex 赋值为 'M'
    stu_1.sex = 'M';
    // 给 stu_1 的成员 score 赋值为 89.5
    stu_1.score = 89.5;
    // 格式化输出 stu_1 的各个成员信息
    printf("No. :%ld\nname:%s\nsex:%c\nscore:%5.1f\n", stu_1.num, stu_1.name, stu_1.sex,
stu_1.score);
    // 通过指针 p 间接访问并格式化输出 stu_1 的各个成员信息
    // (*p) 表示指针 p 所指向的结构体变量，通过 (*p).成员名 来访问结构体成员
    printf("\nNo.:%ld\nname:%s\nsex:%c\nscore:%5.1f\n", (*p).num, (*p).name, (*p).sex,
(*p).score);
    // 程序正常结束，返回 0
    return 0;
}
```

```
No. :10101
name:Li Lin
sex:M
score: 89.5

No.:10101
name:Li Lin
sex:M
score: 89.5
```

（4）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#define LEN sizeof(struct Student)
// 定义结构体 Student，用于存储学生的信息
struct Student {
    long num;  // 存储学生的学号
    float score;  // 存储学生的分数
    struct Student* next;  // 指向下一个 Student 结构体的指针，用于构建链表
};
int n;  // 用于记录学生的数量
// 函数 creat 用于创建一个存储学生信息的链表
struct Student* creat(void)
{
    struct Student* head;  // 链表的头指针
    struct Student* p1, * p2;  // p1 用于创建新节点，p2 用于指向当前链表的最后一个节点
    n = 0;  // 初始化学生数量为 0
    p1 = p2 = (struct Student*)malloc(LEN);  // 为第一个节点分配内存空间
    scanf("%ld,%f", &p1->num, &p1->score);  // 从用户输入中读取学生的学号和分数，注意输入格式为“学号,分数”
    head = NULL;  // 初始时，头指针置为 NULL
    while (p1->num != 0)
    {  // 当输入的学号不为 0 时，继续添加节点
        n = n + 1;  // 学生数量加 1
        if (n == 1) head = p1;  // 如果是第一个节点，将其设置为头节点
        else p2->next = p1;  // 否则将新节点添加到链表末尾
        p2 = p1;  // p2 指向当前的最后一个节点
        p1 = (struct Student*)malloc(LEN);  // 为下一个节点分配内存空间
        scanf("%ld,%f", &p1->num, &p1->score);  // 继续读取下一个节点的学号和分数
    }
    p2->next = NULL;  // 最后一个节点的 next 指针置为 NULL，结束链表
    return(head);  // 返回链表的头指针
}
int main()
{
    struct Student* pt;  // 定义一个指针用于遍历链表
    pt = creat();  // 调用 creat 函数创建链表
    printf("\nnum\tscore\n");  // 输出表头
    while (pt != NULL)
    {  // 当链表不为空时
        printf("%ld\t%5.1f\n", pt->num, pt->score);  // 输出学生的学号和分数
        pt = pt->next;  // 指针指向下一个节点
    }
```

```
    return 0;
}
```

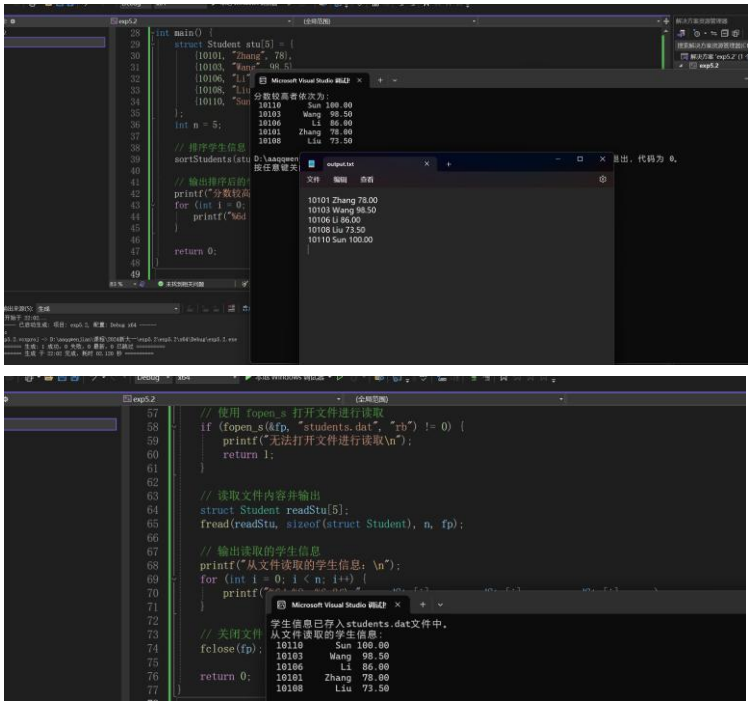
```
1001,85.5
1002,90.0
1003,88.0
0,0

num       score
1001       85.5
1002       90.0
1003       88.0
```

（5）

```
#include <stdio.h>
#include <string.h>
// 定义结构体 Student
struct Student {
    int num;
    char name[20];
    double score;
};
// 按成绩从高到低排序
void sortStudents(struct Student stu[], int n) {
    struct Student temp;
    for (int i = 0; i < n - 1; i++) {
        int k = i;
        for (int j = i + 1; j < n; j++) {
            if (stu[j].score > stu[k].score) {
                k = j;
            }
        }
        temp = stu[k];
        stu[k] = stu[i];
        stu[i] = temp;
    }
}
// 主函数
int main() {
```

```
    struct Student stu[5] = {
        {10101, "Zhang", 78},
        {10103, "Wang", 98.5},
        {10106, "Li", 86},
        {10108, "Liu", 73.5},
        {10110, "Sun", 100}
    };
    int n = 5;
    // 排序学生信息
    sortStudents(stu, n);
    // 使用 fopen_s 打开文件
    FILE* fp;
    if (fopen_s(&fp, "students.dat", "wb") != 0) {
        printf("无法打开文件\n");
        return 1;
    }
    // 将排序后的学生信息写入文件
    fwrite(stu, sizeof(struct Student), n, fp);
    // 关闭文件
    fclose(fp);
    // 输出提示信息
    printf("学生信息已存入students.dat文件中。\n");
    // 使用 fopen_s 打开文件进行读取
    if (fopen_s(&fp, "students.dat", "rb") != 0) {
        printf("无法打开文件进行读取\n");
        return 1;
    }
    // 读取文件内容并输出
    struct Student readStu[5];
    fread(readStu, sizeof(struct Student), n, fp);
    // 输出读取的学生信息
    printf("从文件读取的学生信息：\n");
    for (int i = 0; i < n; i++) {
        printf("%6d %8s %6.2f\n", readStu[i].num, readStu[i].name, readStu[i].score);
    }
    // 关闭文件
    fclose(fp);
    return 0;
}
```



4. 上机遇到的问题及解决办法记录以及体会

“C 语言程序设计实验”实验报告

实验 6—综合应用

1. 试验目的

（1） 综合应用，较复杂的应用案例。

2. 实验内容

（1） 电话订餐信息处理，数据包括姓名、人数、用餐时间、编号，功能为数据的插入、查询、修改、删除、显示；采用字符式菜单。《C 程序设计学习辅导》案例 3 “数组版” 与“链表版” 任选。

3.程序清单及运行结果 (清单[可打印可手写]有缩进，代码整洁、层次清晰；要有注释**[必须手写]**；而且要求画出模块调用关系图**[必须手绘]**)

```
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
#include<string.h>
#include<stdlib.h>

// 定义一个结构体 information 存储相关信息
struct information {
    char name[8];
    int sum;
    char time[8];
    int number;
    struct information* next;  // 指向下一个信息节点的指针，用于构建链表
};

// 函数声明
void Insert(struct information** pHead);
void Search(struct information* Head);
void Update(struct information* Head);
void Delete(struct information** pHead);
void Show(struct information* Head);
void DeleteAll(struct information* Head);

int main()
{
    int i;
    struct information* hd = NULL;
    do {
```

```
        printf("\n");
        printf("1------插入(Insert)\n");
        printf("2------查询(Search)\n");
        printf("3------修改(Update)\n");
        printf("4------删除(Delete)\n");
        printf("5------显示(Show)\n");
        printf("6------退出(Exit)\n");
        scanf("%d", &i);
        switch (i)
        {
        case 1: Insert(&hd);  // 调用插入函数
            break;
        case 2: Search(hd);  // 调用查询函数
            break;
        case 3: Update(hd);  // 调用更新函数
            break;
        case 4: Delete(&hd);  // 调用删除函数
            break;
        case 5:Show(hd);  // 调用显示函数
            break;
        case 6: DeleteAll(hd);  // 调用删除全部函数
            break;
        default: printf("错误选择！ 请重选\n"); break;
        }
    } while (i != 6);

    return 0;
}

// 插入新的信息节点
void Insert(struct information** pHead)
{
    struct information* a, * b, * c;
    int num;
    printf("请输入编号\n");
    scanf("%d", &num);
    a = b = *pHead;
    // 检查是否已经存在相同编号
    while (a != NULL)
        if (a->number == num)
        {
            printf("已经有相同的编号\n"); return;
        }
        else { b = a; a = a->next; }
```

```
    c = (struct information*)malloc(sizeof(struct information));  // 为新节点分配内存
    if (c == NULL) { printf("分配空间失败！\n"); return; }
    c->next = NULL;
    if (b == NULL)
        *pHead = c;  // 如果链表为空，将新节点作为头节点
    else
        b->next = c;  // 否则将新节点添加到链表末尾
    c->number = num;

    printf("请输入姓名：\n");
    scanf("%s", c->name);
    printf("请输入人数：\n");
    scanf("%d", &c->sum);
    printf("请输入用餐时间：\n");
    scanf("%s", c->time);
}

// 查找特定编号的信息节点
void Search(struct information* Head)
{
    int number, flag = 1;
    struct information* p;
    printf("请输入要查询的编号：\n");
    scanf("%d", &number);
    p = Head;
    while (p != NULL && flag)
        if (p->number == number)
        {
            // 输出找到的信息
            printf("\n姓名：%s, ", p->name);
            printf("人数：%d, ", p->sum);
            printf("用餐时间：%s\n", p->time);
            flag = 0;
        }
        else
            p = p->next;
    if (flag == 1) printf("没有查询到!\n");

}

// 更新特定编号的信息节点
void Update(struct information* Head)
{
    int number, flag = 1;
```

```
    struct information* p;
    printf("请输入要修改数据的编号: \n");
    scanf("%d", &number);
    p = Head;
    while (p != NULL && flag)
        if (p->number == number)
        {
            // 修改节点的信息
            printf("请输入人数:");
            scanf("%d", &p->sum);
            printf("请输入用餐时间: ");
            scanf("%s", p->time);
            flag = 0;
        }
        else
            p = p->next;
    if (flag == 1) printf("没有查询到可以修改的数据!\n");

}

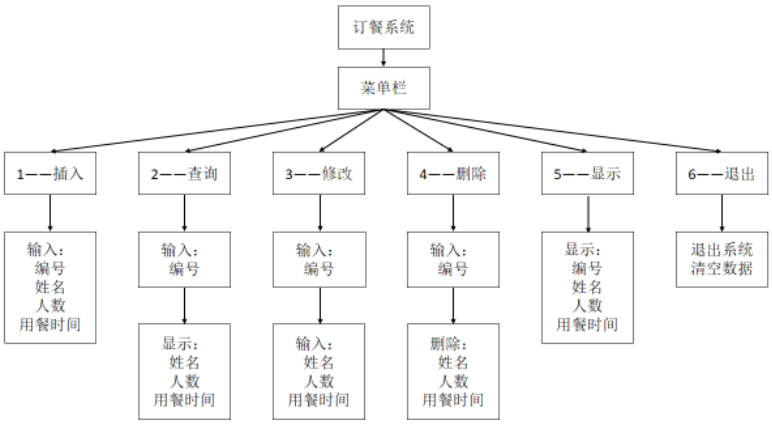
// 删除特定编号的信息节点
void Delete(struct information** pHead)
{
    int number, flag = 1;
    struct information* p, * q;
    printf("请输入要删除数据的编号: \n");
    scanf("%d", &number);
    p = q = *pHead;
    while (p != NULL && flag)
        if (p->number == number)
        {
            if (p == *pHead)
            {
                *pHead = p->next; free(p);  // 删除头节点
            }
            else
            {
                q->next = p->next; free(p);  // 删除非头节点
            }
            flag = 0;
        }
        else
        {
            q = p; p = p->next;
```

```
        }
    if (flag == 1) printf("没有查询到可以删除的数据!\n");
}

// 显示所有信息节点的信息
void Show(struct information* Head)
{
    struct information* p;
    p = Head;
    while (p != NULL)
    {
        // 输出节点信息
        printf("姓名：%-10s", p->name);
        printf("人数：%-10d", p->sum);
        printf("用餐时间：%-10s", p->time);
        printf("编号：%-10d\n", p->number);
        p = p->next;
    }

}

// 删除链表中所有节点
void DeleteAll(struct information* Head)
{
    struct information* p, * q;
    p = Head;
    while (p != NULL)
    {
        q = p->next;
        free(p);  // 释放节点内存
        p = q;
    }
}
```



```
1------插入(Insert)
2------查询(Search)
3------修改(Update)
4------删除(Delete)
5------显示(Show)
6------退出(Exit)
1
请输入编号
1
请输入姓名:
1
请输入人数:
1
请输入用餐时间:
1

1------插入(Insert)
2------查询(Search)
3------修改(Update)
4------删除(Delete)
5------显示(Show)
6------退出(Exit)
2
请输入要查询的编号:
1

姓名: 1, 人数: 1, 用餐时间: 1

1------插入(Insert)
2------查询(Search)
3------修改(Update)
4------删除(Delete)
5------显示(Show)
6------退出(Exit)
3
请输入要修改数据的编号:
1
请输入人数:2
请输入用餐时间: 3

1------插入(Insert)
2------查询(Search)
3------修改(Update)
4------删除(Delete)
5------显示(Show)
6------退出(Exit)
4
请输入要删除数据的编号:
```

4. 上机遇到的问题及解决办法记录以及体会